

Additive Genetic Models in Mixed Populations

Facundo Muñoz

2026-09-25 breedR version: 0.13.0

Contents

Full diallel trial with founders from two different base populations E and J

```
## Setup
library(breedR)
library(ggplot2)
set.seed(123)

## Simulation parameters
n(founders) <- c(E = 9, J = 9)
sigma2 <- c(E = 3, J = 2, resid = 1)

founders <- data.frame(id = c(paste0('E', 1:n(founders['E'])),
                             paste0('J', 1:n(founders['J']))),
                      pop.idx = c(rep(1, n(founders['E'])),
                                 rep(2, n(founders['J']))),
                      BV = c(rnorm(n(founders['E']), sd = sqrt(sigma2['E'])),
                           rnorm(n(founders['J']), sd = sqrt(sigma2['J']))))

n.obs <- sum(n(founders))*150
obs.parents.idx <- matrix(sample(nrow(founders), 2*n.obs, replace = TRUE), ncol = 2)

## The 'family' is independent of the order of the parents
## While the population only takes into account the origin
## e.g. cross2fam(c('J3', 'E1')) gives 'E1:J3'
## while cross2pop(c('J3', 'E1')) gives 'EJ'
cross2fam <- function(x) paste(founders$id[sort(x)], collapse = ':')
cross2pop <- function(x) paste(names(n(founders))[founders$pop.idx[sort(x)]], collapse = '')

## Mendelian sampling term
msp <- function(x) {
  ss <- sigma2[founders$pop.idx[x]]
  s2 <- (ss[1] + ss[2])/4
  rnorm(1, sd = sqrt(s2))
}

dat <- data.frame(
  id = sum(n(founders)) + seq.int(n.obs),
  dad = obs.parents.idx[, 1],
  mum = obs.parents.idx[, 2],
```

```

fam = as.factor(apply(obs.parents.idx, 1, cross2fam)),
sp  = as.factor(apply(obs.parents.idx, 1, cross2pop)),
bv  = apply(obs.parents.idx, 1,
            function(x) mean(founders$BV[x]) + msp(x)),
resid = rnorm(n.obs, sd = sqrt(sigma2['resid']))

dat <- transform(dat,
                 y = bv + resid)

## Printing simulated setting
print(table(dat[, c('mum', 'dad')]), zero.print = "")

```

```

##      dad
## mum  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
##  1   8 11 12  7  7 14  8 10  9 14 11 13  7  8  8  7 11  7
##  2  12  8  5 12  5  9  5 10  7 13  7  5  5  7  8 14 11  7
##  3   6 12  5  7  8 10  6 13  5  5  7  6  6  9  9  7  2 13
##  4   6  8  8  6 11  8 11  9 11 10  9  5  8  6  9  9 10 11
##  5   7  7  8  4 13  8 14  6  6  5  6 11  7  5  3  6  8 11
##  6  10  5  9  7  6  6  5  7 14  9  5  9  4  6  2  7  9  7
##  7   8 10  7 14 14 10  8 11  5  8  9  5  9 15  6  7  7  8
##  8  10  7 11  5  5  3  3 15 10 10  7  9  7  7  9  4  8 11
##  9   6  5  9  2  3 11 11  7  7  5 11 10  8  8  9  9  5  4
## 10   6  2 10  9  7  4 11 13  6  9  7  7  6 14 14  7 13 10
## 11   4  3  9 10  7 10  5 14  9  9 12  7  8  5  9  8 11  7
## 12   9  5 10  5 10  6  6  6  8  9 11  7 10  8 10  8  6 10
## 13   3 14  8  8  5  5  9  9  5  6 10 10  7  6  8  8  8  5
## 14   5  9  8  4 12 10  8  9  8 17 12 13  4  7  8 11  9  8
## 15   6 11  6 12 14  7 12  5 17 12  6  4  8  9 12 10  3 13
## 16  13 10  9  8  9  8  8 10  9 11 10  8  4  7  6  9 10  5
## 17   7  5  7  8 13 11 10  6  6 14  9  9 14 11  9  4  9  8
## 18   9  9  8 11  9  4 12 14  9  4 11 15  5 10 10  8 12 10

```

```
str(dat)
```

```

## 'data.frame':    2700 obs. of  8 variables:
## $ id      : num  19 20 21 22 23 24 25 26 27 28 ...
## $ dad     : int  11 7 12 15 10 13 7 9 9 10 ...
## $ mum     : int  7 11 5 16 17 17 8 3 8 14 ...
## $ fam     : Factor w/ 171 levels "E1:E1","E1:E2",...: 98 98 74 163 134 155 95 42 107 131 ...
## $ sp      : Factor w/ 3 levels "EE","EJ","JJ": 2 2 2 3 3 3 1 1 1 3 ...
## $ bv      : num  2.797 0.395 0.19 1.284 0.862 ...
## $ resid   : num  -0.721 -2.585 0.271 -0.142 1.104 ...
## $ y       : num  2.076 -2.19 0.461 1.142 1.966 ...

```

There will be two independent additive-genetic variance models for the J and E populations. Each variance component will be estimated using the *pure* offspring only.

```

## Build a pedigree for the whole mixed population
## and get the kinship matrix A
ped <- build_pedigree(1:3, data = dat)
A <- pedigreeemm::getA(ped)

```

```

## 'as(<dtTMatrix>, "dtCMatrix")' is deprecated.
## Use 'as(., "CsparseMatrix")' instead.
## See help("Deprecated") and help("Matrix-deprecated").

```

```
## Build the full incidence matrix
Z <- as(dat$id, 'indMatrix')

## Give the index vector of additive-genetic random effects
## that belong to one subpopulation;
## 'E', 'J' (founders) or 'EE', 'EJ' or 'JJ' (offspring).
idx_pop <- function(x) {
  if (nchar(x) == 1) grep(x, founders$id)
  else
    match(dat$id[dat$sp == x], as.data.frame(ped)$self)
}
```

0.1 Method 1: Hybrids as an independent population

This is the easiest way to go. It works as a first approximation, but has several shortcomings.

It needs to estimate one *virtual* variance for the hybrid population which is not linked to any genetic variance in the *real world*. Moreover, we don't use the hybrid observations to learn about the two variances that really matter: σ^2_E and σ^2_J

The advantage is that it predicts the Breeding Values of the hybrid offspring. However, the accuracy may be limited by the violation of the *single population* hypothesis of the model.

```
## Avoid estimating BLUPS for which we don't have information
## Otherwise, the run takes much longer (5 hs vs 6 min in this example)

## A[idx_pop('EE'), idx_pop('JJ')] # This is null: populations are independent
Z_EE <- Z[, idx_pop('EE')]
Z_EJ <- Z[, idx_pop('EJ')]
Z_JJ <- Z[, idx_pop('JJ')]

A_EE <- A[idx_pop('EE'), idx_pop('EE')]
A_EJ <- A[idx_pop('EJ'), idx_pop('EJ')]
A_JJ <- A[idx_pop('JJ'), idx_pop('JJ')]

## Now fit a model with three additive-genetic components,
## by means of generic effects (as only one 'genetic' is allowed in breedR)

res1 <- remlf90(y ~ sp,
  generic = list(
    E = list(incidence = Z_EE,
              covariance = A_EE),
    J = list(incidence = Z_JJ,
              covariance = A_JJ),
    H = list(incidence = Z_EJ,
              covariance = A_EJ)),
  data = dat
)

## Using default initial variances given by default_initial_variance()
## See ?breedR.getOption.

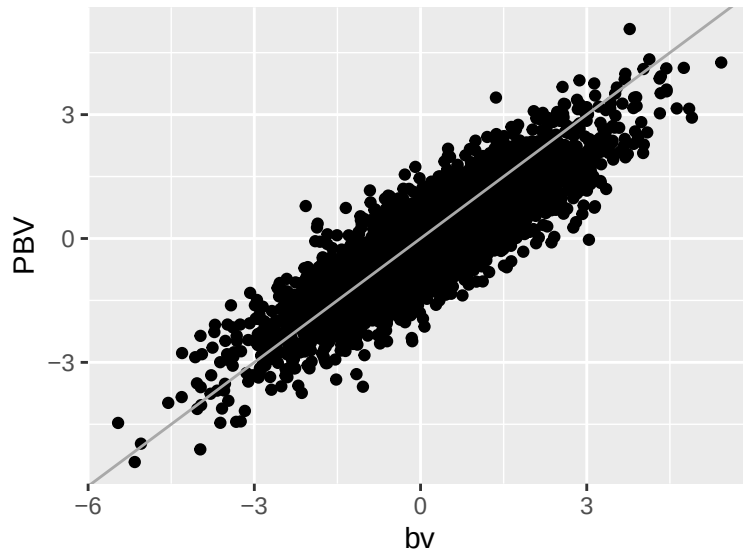
summary(res1)

## Formula: y ~ 0 + sp
## Data: dat
```

```
##   AIC   BIC logLik
##  9937 9961  -4965
##
## Parameters of special components:
##
##
## Variance components:
##           Estimated variances   S.E.
## E                2.7194 0.7185
## J                2.5068 0.7144
## H                2.7841 0.6992
## Residual          0.8589 0.3495
##
## Fixed effects:
##           value   s.e.
## sp.EE 0.23745 0.5527
## sp.EJ 0.20826 0.3955
## sp.JJ 0.21205 0.5306

PBV <- as.matrix(cbind(Z_EE, Z_JJ, Z_EJ)) %*%
  do.call('rbind', lapply(ranef(res1), function(x) cbind(PBV = x, se = attr(x, 'se'))))

ggplot(cbind(dat, PBV), aes(bv, PBV)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, col = 'darkgray')
```



0.2 Method 2: GCA/SCA model for hybrids

Another approach is to model hybrids with GCAs and SCAs, with additional variance parameters. The additive component of the genetic variances for the E and J populations is half the population variance. But these parameters will also gather dominance and epistasis effects.

However, we don't take advantage of the known relationship between the additive components by treating them as independent parameters. Furthermore, we don't use the relationship between hybrids and pures to learn about the original genetic variances. Nor even within hybrids, as we treat SCA as an unstructured effect, while there are both half and full siblings.

```

## We only want to apply 'dad', 'mum' and 'sca' effects to hybrids,
## and make it zero for non-hybrids. We do so by pre-multiplying by a
## diagonal indicator matrix
Ind <- diag(dat$sp == 'EJ')

## Build a specific incidence matrices for generic random effects
Z_dad <- Ind %*% as(dat$dad, 'indMatrix')
Z_mum <- Ind %*% as(dat$mum, 'indMatrix')
Z_sca <- Ind %*% as(as.numeric(dat$fam), 'indMatrix')

## The structure variances are diagonal
D <- diag(sum(n(founders)))

res2 <- remlf90(y ~ sp,
               generic = list(
                 E = list(incidence = Z_EE,
                          covariance = A_EE),
                 J = list(incidence = Z_JJ,
                          covariance = A_JJ),
                 dad = list(incidence = Z_dad,
                           covariance = D),
                 mum = list(incidence = Z_mum,
                           covariance = D),
                 sca = list(incidence = Z_sca,
                           covariance = diag(nlevels(dat$fam))),
                 data = transform(dat)
               )
)

## Using default initial variances given by default_initial_variance()
## See ?breedR.getOption.

summary(res2)

```

```

## Formula: y ~ 0 + sp
## Data: transform(dat)
## AIC BIC logLik
## 10010 10045 -4999
##
## Parameters of special components:
##
## Variance components:
## Estimated variances S.E.
## E 0.9373800 0.2014200
## J 0.7679600 0.1731200
## dad 0.6773600 0.2457700
## mum 0.7146600 0.2585500
## sca 0.0002231 0.0002437
## Residual 2.0314000 0.0657310
##
## Fixed effects:
## value s.e.
## sp.EE 0.23742 0.3285
## sp.EJ 0.21363 0.2809
## sp.JJ 0.20965 0.2979

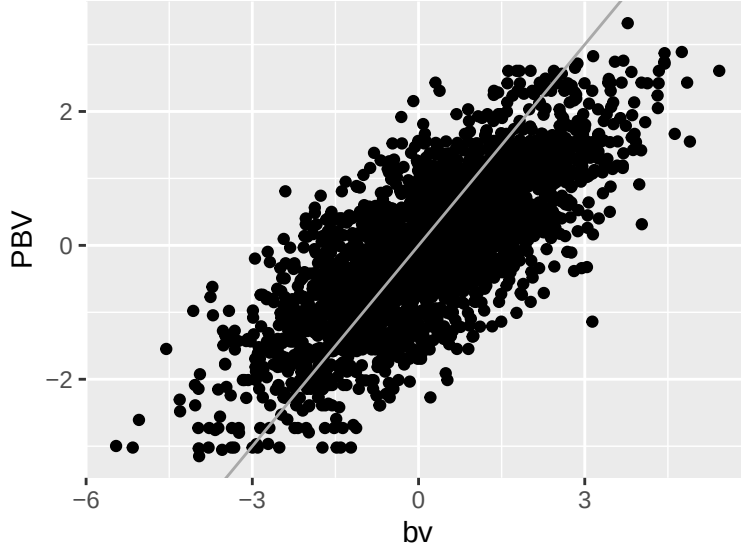
```

```

PBV <- as.matrix(cbind(Z_EE, Z_JJ, Z_dad, Z_mum, Z_sca)) %*%
do.call('rbind', lapply(ranef(res2), function(x) cbind(PBV = x, se = attr(x, 'se'))))

ggplot(cbind(dat, PBV), aes(bv, PBV)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, col = 'darkgray')

```



This approach does not work very well. Much of the variation has not been adequately accounted for, and ended up in the residuals. May be there is some misspecification in the model.

0.3 Method 3: grid search

The two variances enter the covariance of the mixture linearly, once the relationships are split by the population they come through. Following García-Cortés and Toro (2006), write $A = A_E + A_J$, where A_E (A_J) accumulates only the relationships passing through founders of population E (J), together with that population's share of each Mendelian sampling term. Splitting the matrix in blocks corresponding to each subpopulation, the covariance matrix for the mixture is

$$\begin{aligned}
\Sigma = & \begin{bmatrix} \sigma_E^2 A_{E:E} & \mathbf{0} & \sigma_E^2 A_{E:EE} & \sigma_E^2 A_{E:H} & \mathbf{0} \\ \mathbf{0} & \sigma_J^2 A_{J:J} & \mathbf{0} & \sigma_J^2 A_{J:H} & \sigma_J^2 A_{J:JJ} \\ \sigma_E^2 A_{E:EE} & \mathbf{0} & \sigma_E^2 A_{EE} & \sigma_E^2 A_{EH} & \mathbf{0} \\ \sigma_E^2 A_{E:H} & \sigma_J^2 A_{J:H} & \sigma_E^2 A_{HE} & \sigma_E^2 A_{HH}^E + \sigma_J^2 A_{HH}^J & \sigma_J^2 A_{HJ} \\ \mathbf{0} & \sigma_J^2 A_{J:JJ} & \mathbf{0} & \sigma_J^2 A_{JH} & \sigma_J^2 A_{JJ} \end{bmatrix} \\
& = \sigma_E^2 (A_E + \lambda A_J),
\end{aligned} \tag{1}$$

where $\lambda = \frac{\sigma_J^2}{\sigma_E^2}$, and A_{HH}^E , A_{HH}^J are the parts of the hybrid-hybrid relationships passing through E and through J founders. Every relationship between a hybrid and a pure E individual passes through E founders, so those blocks keep the factor one, and the pure J ones carry λ . Within the hybrids no single factor will do: full sibs give $\frac{1+\lambda}{2} A_{HH}$, but half sibs are related through E or through J, depending on which parent they share.

Σ is linear in σ_E^2 and σ_J^2 , so both could be estimated at once with two effects having covariance matrices A_E and A_J . Here we take the simpler route of profiling the ratio λ , which needs a single variance component per fit.

However, if we are not interested in evaluating the parents, we can disregard the first two block rows and columns from the matrix.

Now, fit the model for several values of λ and maximize the likelihood.

```
## Setup parallel computing
# library(doParallel)
# cl <- makeCluster(2)
# registerDoParallel()
# on.exit(stopCluster(cl))

## Introduce the corresponding scaling factors
## in the relationship matrix.
## A = A_E + A_J splits each relationship according to the population it
## passes through, so that Sigma = sigma2_E (A_E + lambda A_J).
## A single factor per block would not do: two hybrids that share one parent
## are related through E or through J, depending on which parent it is.
scale_A <- function(x) {

  ped.df <- as.data.frame(ped)
  fnd <- match(seq_len(nrow(founders)), ped.df$self) ## founder rows
  off <- match(dat$id, ped.df$self) ## offspring rows

  ## Contribution of each founder to each individual of the pedigree
  P <- Matrix::sparseMatrix(
    i = c(fnd, off, off),
    j = c(seq_len(nrow(founders)), dat$dad, dat$mum),
    x = c(rep(1, nrow(founders)), rep(1/2, 2*nrow(dat))),
    dims = c(nrow(A), nrow(founders)))

  ## Relationships passing through the founders of one population only.
  ## Each parent passes on half of its contribution through the Mendelian
  ## sampling term; here every parent is a founder.
  partial_A <- function(pop) {
    Q <- P[, idx_pop(pop), drop = FALSE]
    v <- numeric(nrow(A))
    v[off] <- Matrix::rowSums(Q[off, , drop = FALSE])/2
    Matrix::tcrossprod(Q) + Matrix::Diagonal(x = v)
  }

  ## The J part is scaled by lambda = sigma2_J / sigma2_E
  S <- partial_A('E') + x * partial_A('J')
  dimnames(S) <- dimnames(A)
  S
}

## Conditional likelihood given lambda
cond_lik <- function(x) {
  require(breedR)
  ## Conditional structure matrix
  S <- scale_A(x)

  ## Temporarily, let's use only the pure pops
  idx <- c(idx_pop('EE'), idx_pop('JJ'))

  suppressWarnings(
    res <- remlf90(y ~ sp,
```

```

        generic = list(
          E = list(incidence = Z[dat$sp != 'EJ', idx],
                   covariance = S[idx, idx])),
          data = dat[dat$sp != 'EJ', ]
        )
      )
      logLik(res)
    }

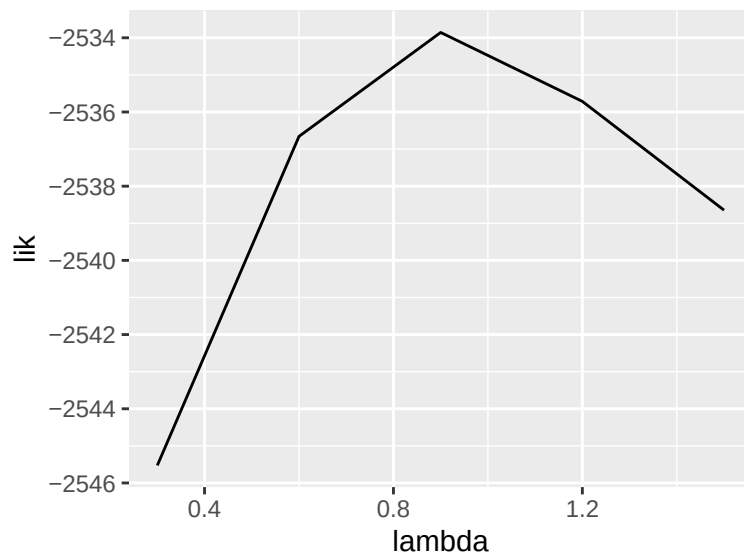
lambda <- seq(.3, 1.5, length.out = 5)

lik <- sapply(lambda, cond_lik) # (sequential)

## Using default initial variances given by default_initial_variance()
## See ?breedR.getOption.
##
## Using default initial variances given by default_initial_variance()
## See ?breedR.getOption.
##
## Using default initial variances given by default_initial_variance()
## See ?breedR.getOption.
##
## Using default initial variances given by default_initial_variance()
## See ?breedR.getOption.
##
## Using default initial variances given by default_initial_variance()
## See ?breedR.getOption.
# lik <- foreach(x = seq.int(lambda), .combine = c) %dopar% cond_lik(lambda[x])

ggplot(data.frame(lambda, lik), aes(lambda, lik)) +
  geom_line()

```



The log-likelihood peaks inside the grid, and the fit below uses the best grid point. The estimate is only as fine as the grid, and if the best point lands on either end, the grid needs widening. With nine founders per population λ is estimated imprecisely. The same simulation run under R's pre-3.6 sampler

(`RNGkind(sample.kind = "Rounding")`) puts the best grid point at 0.6 rather than 0.9. The simulated value is $\lambda = 2/3$.

The profile uses only the pure offspring, whose blocks reduce to A_{EE} and λA_{JJ} . At the simulated λ the scaled matrix matches the covariance of the simulated breeding values exactly, hybrids included.

The hybrids are left out of the fits, but they do carry information on both variances: hybrids that share their E parent are related through σ_E^2 , and those that share their J parent through σ_J^2 .

```
## Take lambda maximizing the likelihood
lambda0 <- lambda[which.max(lik)]

S <- scale_A(lambda0)

## Temporarily, let's use only the pure pops
idx <- c(idx_pop('EE'), idx_pop('JJ'))

# ## Remove the founders, which I don't want to evaluate
# idx <- -(1:sum(n_founders))

res3 <- remlf90(y ~ sp,
               generic = list(
                 E = list(incidence = Z[dat$sp != 'EJ', idx],
                           covariance = S[idx, idx])),
               data = dat[dat$sp != 'EJ', ])

## Using default initial variances given by default_initial_variance()
## See ?breedR.getOption.

summary(res3)

## Formula: y ~ 0 + sp
##   Data: dat[dat$sp != "EJ", ]
##   AIC   BIC logLik
##  5072 5082  -2534
##
## Parameters of special components:
##
## Variance components:
##           Estimated variances   S.E.
## E                2.642 0.9102
## Residual          0.913 0.4380
##
## Fixed effects:
##           value   s.e.
## sp.EE 0.23745 0.5449
## sp.EJ 0.00000 0.0000
## sp.JJ 0.21201 0.5168

PBV <- as.matrix(Z[dat$sp != 'EJ', idx]) %*%
  do.call('rbind', lapply(ranef(res3), function(x) cbind(PBV = x, se = attr(x, 'se'))))

ggplot(cbind(dat[dat$sp != 'EJ', ], PBV), aes(bv, PBV)) +
```

```
geom_point() +  
geom_abline(intercept = 0, slope = 1, col = 'darkgray')
```

